

Hệ thống đấu giá trực tuyến

Online Auction System | Team N7 | UET - VNU.

Phần 1: Mục tiêu, phạm vi thực hiện.

Mục tiêu Học thuật

- Ứng dụng nguyên lý OOP nâng cao và Design Patterns bao gồm Singleton, Factory, Observer, và Command.
- Lập trình Socket đa luồng với mô hình Client-Server phân tán.
- Thiết kế kiến trúc phân lớp rõ ràng bao gồm Presentation, Business Logic, và Data Access.

Mục tiêu Kỹ thuật

- Độ trễ đồng bộ cực thấp dưới 100 ms qua kết nối Socket TCP.
- Bảo mật 2FA theo phương thức TOTP bằng cách quét mã QR qua Google Authenticator.
- An toàn dòng tiền ví theo chuẩn ACID và chống race condition bằng Optimistic Locking.

Phạm vi Thực hiện

- Mô hình Client-Server phân tán — Server giữ SQLite, Client giao tiếp qua WebSocket TCP.

Ba vai trò nghiệp vụ

- Bidder: Đặt giá thủ công, Auto-Bid tự động, xem biểu đồ giá real-time.
- Seller: Đăng tải hàng hóa, thiết lập cấu hình phiên đấu giá.
- Admin: Phê duyệt phiên, giám sát dòng tiền toàn hệ thống.

Công nghệ sử dụng

- Java 17, JavaFX, WebSocket, SQLite, BCrypt, TOTP, Maven, GitHub Actions.

Phần 2: Kiến trúc tổng thể hệ thống.

Thành phần	Mô tả chi tiết
Client (JavaFX MVC)	Giao diện xây dựng bằng JavaFX FXML + CSS. Tách biệt hoàn toàn tầng hiển thị và tầng kết nối thông qua AuctionEventBus — giúp cập nhật UI an toàn, không đồng bộ trên luồng JavaFX. Các Controller nhận event từ Bus và cập nhật Scene Graph; không gọi Socket trực tiếp.
Networking (WebSocket)	WebSocket TCP thay thế Socket truyền thống. Gói tin định dạng JSON được mã hóa AES-256-GCM. Client gửi Command, Server trả Response qua cùng kênh full-duplex. Batching + Debouncing giảm tải truyền real-time cho broadcast giá thầu đồng thời.
Server (Multi-Thread)	MultiThreadedServer đón nhận kết nối và chia luồng ClientHandler xử lý song song. CommandDispatcher định tuyến gói tin JSON đến các Handler chuyên biệt. AutoBidEngine chạy nền đặt giá tự động. Publish-Subscribe pattern phát broadcast cập nhật giá tới tất cả Client trong phiên.
Database (SQLite + DAO)	Đóng gói toàn bộ câu lệnh SQL qua lớp DAO (Data Access Object). TransactionManager quản lý giao dịch ACID đảm bảo nhất quán dữ liệu. Optimistic Locking qua cột version chống race condition khi nhiều luồng ghi đồng thời.

Phần 3: Chức năng đạt được theo Barem.

Chức năng / Tiêu chí	Hướng giải quyết kỹ thuật	Lý do lựa chọn
----------------------	---------------------------	----------------

Thiết kế lớp & cây kế thừa	Entity (abstract) → Item → Electronics/Art/Vehicle; User → Bidder/Seller/Admin; Auction, BidTransaction.	Phân cấp rõ ràng, mở rộng loại Item/vai trò mới không cần sửa code cũ – tuân thủ Open/Closed.
Áp dụng nguyên tắc OOP	Encapsulation: trường private + getter/setter. Inheritance: Item/User → các lớp con. Polymorphism: override printInfo(), validate(). Abstraction: abstract class + interface cho hành vi chung.	Giảm coupling, dễ kiểm thử từng lớp độc lập, tái sử dụng logic xuyên suốt hệ thống.
Design Pattern áp dụng	Singleton: DatabaseConnection, AuctionManager. Factory Method: ItemFactory tạo Electronics/Art/Vehicle. Observer: AuctionEventBus phát bid mới tới Client. Command: BidCommand, FetchAuctionCommand đóng gói lệnh mạng.	Singleton tránh tạo nhiều kết nối DB. Factory Method thêm loại Item mới không sửa code cũ. Observer tách Server khỏi UI. Command dễ log và test độc lập.
Xử lý lỗi & ngoại lệ	Server validate: giá thầu thấp, phiên đã đóng (FINISHED/CANCELED), số dư thiếu → trả mã lỗi JSON. Client hiển thị Toast notification. Reconnect tự động khi mất kết nối.	Validate server-side ngăn dữ liệu bẩn dù Client bị can thiệp. Toast giúp người dùng hiểu lỗi thay vì crash im lặng.
Đăng ký / Đăng nhập người dùng	BCrypt hash mật khẩu. JWT session token lưu phía Client. Kiểm tra credentials qua LoginHandler Server.	BCrypt là chuẩn băm bảo mật chống brute-force hiệu quả nhất hiện nay. JWT giúp xác thực stateless, dễ scale.
Quản lý người dùng (Admin)	CRUD người dùng qua AdminUserHandler. Block/Unblock kèm cơ chế Kick phiên làm việc ngay lập tức. Ngăn Admin tự cấm chính mình hoặc xóa Admin khác.	Bảo đảm phân quyền chặt chẽ: người bị cấm không giữ kết nối sống – ngăn bypass quyền Admin.
Đăng tải sản phẩm (Seller)	Seller tạo phiên đấu giá, upload ảnh (nén phía Client trước khi gửi lên Cloudinary). Admin phê duyệt mỗi sản phẩm được đăng.	Nén ảnh phía Client giảm băng thông truyền. Quy trình duyệt Admin đảm bảo chất lượng sản phẩm.
Đặt giá thủ công (Manual Bidding)	Client gửi lệnh BID kèm giá trị. Server validate: giá > current_price + step, người dùng có đủ số dư, phiên đang RUNNING. Optimistic Locking cập nhật DB.	Kiểm tra server-side ngăn giá giả mạo từ Client. Optimistic Lock đảm bảo chính xác khi có hàng nghìn luồng đặt giá đồng thời.
Đặt giá tự động (Auto-Bid Engine)	AutoBidEngine chạy nền theo Publish-Subscribe. Khi có giá mới, Engine tự động đặt giá tăng lên max_bid của từng user đã cấu hình. Sử dụng hàng đợi ưu tiên (Priority Queue).	Engine tách biệt khỏi Controller ngăn deadlock. Priority Queue đảm bảo Bot ưu tiên cao nhất phản hồi trước.
Đồng bộ thời gian thực (Real-time Sync)	Broadcast WebSocket phát cập nhật giá tới tất cả Client trong phiên. Debouncing gộp các sự kiện trong 50ms thành 1 gói tin. AuctionEventBus phân phối event trên luồng JavaFX.	Debouncing tránh bão hoặc nghẽn mạng khi có nhiều bid đồng thời. EventBus giữ UI không bị block bởi I/O mạng.
Đấu giá đồng thời – Chống Race Condition	Optimistic Locking qua cột version trên bảng auctions. Nếu version không khớp khi UPDATE, giao dịch bị rollback và Client nhận thông báo lỗi. TransactionManager đảm bảo ACID.	SQLite không hỗ trợ tốt Pessimistic Lock đa luồng. Optimistic Lock tối ưu hiệu năng ghi, ngăn tuyệt đối lỗi

		Lost Update mà không gây ngheñ cổ chai.
Giao dịch Ví điện tử (Wallet/ACID)	Khi đặt giá: trừ available_balance, cộng locked_balance. Khi bị vượt: hoàn tiền atomic. Thanh toán cuối: chuyển locked -> paid. Tích hợp PayPal Sandbox API thực tế.	ACID đảm bảo không mất tiền dù Server crash giữa luồng. PayPal thay thế mock payment bằng giao dịch ngân hàng thực.
Bảo mật 2FA – TOTP	Server sinh khóa bí mật TOTP, mã hóa QR Code. Client hiển thị QR để quét bằng Google Authenticator. Mỗi lần đăng nhập: nhập OTP 6 chữ số, Server xác thực bằng thuật toán HMAC- SHA1.	TOTP là chuẩn RFC 6238 quốc tế, chống tấn công brute-force mật khẩu. Không lưu OTP phía Server tránh rò rỉ.
Anti-Snipping (Kéo dài thời gian)	Nếu có bid mới trong 30 giây cuối, Server tự động gia hạn phiên thêm 30 giây và broadcast cập nhật thời gian tới tất cả Client.	Ngăn kẻ gian lặn đặt giá vào giây cuối cùng để ‘cướp’ phiên. Tăng tính công bằng cho người dùng.
Biểu đồ giá động (Line Chart)	Client vẽ Line Chart bằng JavaFX Chart API, dữ liệu cập nhật real-time qua WebSocket. Hiển thị cả lịch sử biến động giá thầu và số dư ví.	Trực quan hóa lịch sử giá giúp Bidder quyết định chiến lược đặt giá hiệu quả hơn.
Tìm kiếm thông minh (Fuzzy Search)	Hỗ trợ chuyển tiếng Việt có dấu sang không dấu. Thuật toán fuzzy matching cho phép sai thứ tự từ, thiếu/thừa ký tự. Bộ lọc theo danh mục (Selector).	Nâng cao trải nghiệm người dùng Việt Nam – không cần gõ đúng chính tả tiếng Việt có dấu để tìm được kết quả.
Maven + Coding Convention + Mã sạch	Maven multi-module (client/server/common). Google Java Style Guide. Refactor tách WalletService, ClientUserController khỏi Controller.	Build reproducible trên mọi máy. Convention giúp cả nhóm đọc code nhau không cần giải thích.
Unit Test – JUnit	Test BidValidator, TransactionManager, AutoBidEngine. CountdownLatch giả lập 10 luồng bid cùng 1ms stress-test Optimistic Locking.	Test song song phát hiện race condition mà test tuần tự bỏ sốt. Đảm bảo refactor không phá vỡ logic nghiệp vụ.
CI/CD – GitHub Actions	Pipeline tự động khi push/PR: mvn test → mvn package → build Fat JAR. Test đỗ chặn merge.	Không có code lỗi vào nhánh chính. Fat JAR chạy được trên mọi máy chỉ với java -jar.

Phần 4: Phân công công việc giữa các thành viên:

Họ và tên	Vai trò chính	Chi tiết công việc và giải pháp kỹ thuật
Cao Dương Lễ	Principal Architect Bảo mật & Xử lý Song song	- Xử lý Race Condition trong PaymentHandler và AutoBidEngine. - Vá lỗ hổng bypass phân quyền Admin, ngăn Seller sửa phiên đã duyệt. - Triển khai Optimistic Locking + ACID TransactionManager. - Xây dựng cơ chế Kick phiên làm việc ngay lập tức khi Admin Block người dùng. - Phát triển tính năng Anti-Snipping, TOTP 2FA, Quên mật khẩu, Rút tiền. - Tối ưu hóa WebSocket với Batching/Debouncing + Publish-Subscribe.

		Viết Unit Test song song bằng CountdownLatch (giả lập 10 luồng / 1ms).
Nguyễn Quang Mạnh	Kỹ sư Tài chính & OOP Core WebSocket	- Tích hợp PayPal Sandbox API: Chống Double-Spending bằng cách kiểm tra PayPal Order ID. - Chuẩn hóa tiền tệ: Chuyển đổi toàn bộ kiểu dữ liệu từ double sang long trên toàn hệ thống để tránh sai số. - Cập nhật tỷ giá ngoại tệ theo thời gian thực (Real-time). - Tái cấu trúc OOP: Chuẩn hóa mô hình phân cấp User -> Bidder / Seller, áp dụng Factory Method cho cấu trúc Item. - Nâng cấp hệ thống: Chuyển đổi từ Socket truyền thống sang WebSocket kết hợp mã hóa AES-256-GCM. - Tối ưu hiệu năng: Triển khai Non-blocking DB query chống hiện tượng NIO Thread Starvation.
Ngô Vũ Đình Khoa	UI/UX Lead Hệ thống Tìm kiếm & Admin	- Phát triển thuật toán Fuzzy search tiếng Việt có dấu và hỗ trợ tìm kiếm khi sai thứ tự từ. - Xây dựng bộ lọc danh mục sản phẩm thông minh (Selector). - Thiết kế tính năng cho Admin: Hủy phiên vi phạm, đánh giá uy tín người dùng, xem lịch sử hoạt động của Seller. - Refactor kiến trúc MVC: Tách biệt WalletService và ClientUserController. - Sửa lỗi Memory Leak trong JavaFX và tối ưu hóa cơ chế đồng bộ thời gian giữa Client-Server (giảm từ 10 phút xuống còn 5 giây). - Trực quan hóa dữ liệu: Vẽ Line Chart hiển thị biến động giá thầu và số dư ví theo thời gian thực. - Tối ưu giao diện: Hỗ trợ Auto-scale tỷ lệ 0.75, tích hợp nén ảnh phía Client, chuyển đổi ngôn ngữ giao diện sang Tiếng Anh.
Khúc Ngọc Minh	Phát triển Giao diện Ban đầu	- Thiết kế cấu trúc giao diện Đăng nhập & Đăng ký ban đầu. - Xây dựng giao diện xem chi tiết sản phẩm dành cho người bán (ItemDetailController). - Tinh chỉnh tỷ lệ scale của các widget trên giao diện người dùng.

Giải trình chênh lệch đóng góp

- Sự phân chia phản ánh chân thực nỗ lực đóng góp chất xám kỹ thuật của từng thành viên vào lõi dự án. Phần Back-end (Concurrency, DB Transaction, Socket Server, Fintech) đòi hỏi độ phức tạp giải thuật và chịu rủi ro cao hơn so với thiết kế giao diện Client. Cả nhóm đồng thuận 100% với bảng phân công này, đảm bảo tính công bằng và minh bạch.

Phần 5: Tự đánh giá theo baren và kết luận.

Tiêu chí	Điểm tối đa	Tự đánh giá	Ghi chú
Thiết kế lớp & cây kế thừa	0.5	0.5	Entity → Item/User → các lớp con chuyên biệt
Áp dụng nguyên tắc OOP	1.0	1.0	Encapsulation, Inheritance, Polymorphism, Abstraction

Design Pattern	1.0	1.0	Singleton, Factory Method, Observer, Command
Quản lý người dùng & sản phẩm	1.0	1.0	CRUD, Block/Kick, upload ảnh, duyệt Admin
Chức năng đấu giá (Manual Bid)	1.0	1.0	Validate server-side, Optimistic Locking
Xử lý lỗi & ngoại lệ	1.0	1.0	Validate server-side, Toast notification, reconnect
Concurrent Bidding (Race Condition)	1.0	1.0	Optimistic Lock + cột version SQLite
Realtime Update (WebSocket)	0.5	0.5	Debouncing, EventBus, AES-256-GCM
Kiến trúc Client-Server	0.5	0.5	BCrypt + JWT Session
Kiến trúc MVC + DAO	0.5	0.5	JavaFX FXML client, Controller-Model-DAO server
Maven + Convention + Unit Test	1.0	1.0	Google Java Style, JUnit, CountdownLatch stress-test
CI/CD GitHub Actions	0.5	0.5	mvn test + mvn package tự động trên mỗi PR
Auto-Bidding	0.5	0.5	Priority Queue, Pub-Sub, chống deadlock
Anti-Sniping	0.5	0.5	Gia hạn 30s khi bid trong 30s cuối
Bid History Visualization	0.5	0.5	JavaFX Line Chart real-time
Các tính năng khác	0.5	0.5	TOPT, Paypal
TỔNG CỘNG	10	10	10đ bắt buộc + 1.5đ nâng cao => 10đ

Kết luận và hướng phát triển.

Bài học Kinh nghiệm

- **Xử lý đa luồng và đồng bộ mạng là hai bài toán phức tạp nhất của dự án:** Nhóm học được rằng Optimistic Locking hiệu quả hơn Pessimistic Lock với SQLite trong môi trường đa luồng.
- **Tối ưu hóa băng thông:** Áp dụng cơ chế Debouncing và Batching chính là chìa khóa vàng để giảm tải cho hệ thống mạng khi có lượng lớn giá thầu được gửi lên đồng thời.
- **Hiện thực hóa lý thuyết:** Chuẩn ACID không chỉ nằm trên sách vở — từng giao dịch tiền tệ nhỏ nhất đều phải được kiểm thử nghiêm ngặt thông qua các kịch bản stress-test bằng CountdownLatch.

Hướng Mở rộng

- **Chuyên dịch kiến trúc:** Nâng cấp hệ thống sang kiến trúc Microservices, tách biệt hoàn toàn các dịch vụ độc lập bao gồm Auction Service, Wallet Service, và Notification Service.
- **Nâng cao bảo mật:** Sử dụng giao thức WSS (WebSocket Secure kết hợp TLS) để đảm bảo toàn bộ kết nối được mã hóa đầu cuối (end-to-end).
- **Mở rộng hạ tầng lưu trữ:** Triển khai hệ quản trị cơ sở dữ liệu PostgreSQL thay thế cho SQLite nhằm đáp ứng tốt khi quy mô người dùng tăng trưởng mạnh.
- **Tự động hóa quy trình:** Tích hợp hệ thống CI/CD hoàn chỉnh với GitHub Actions nhằm tự động hóa các khâu từ build, test, đóng gói Docker image cho đến deploy.